

RetailPulse

Week 01 - Data Engineering - github.com/SaurabhMastud/retailpulse

Architecture -- RetailPulse

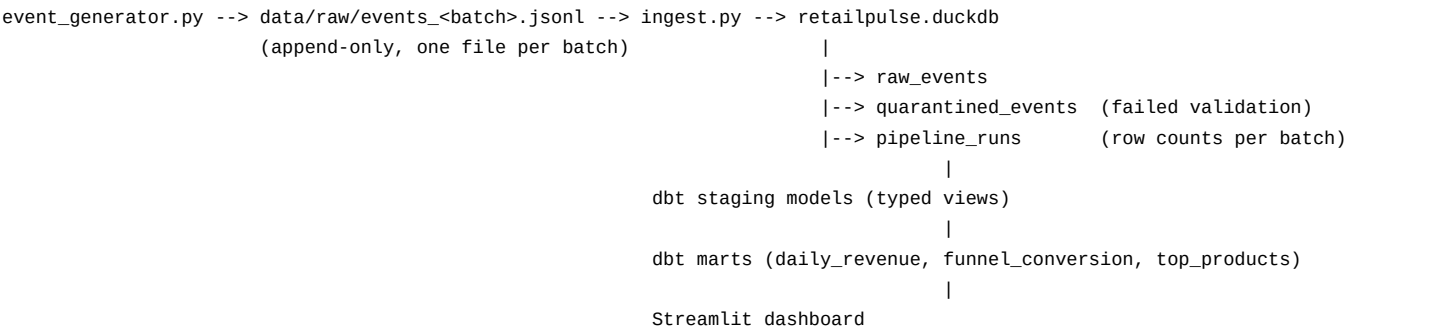
Why this project

Data engineering job postings and modern-data-stack writeups keep converging on the same shape: an event source, a landing zone, transformation-as-code (dbt), an orchestrator, and a thin consumption layer. RetailPulse is a small but complete pass through that whole shape rather than a demo of one link in the chain, using synthetic retail/e-commerce events as the domain because it produces intuitive, easy-to-validate analytics (revenue, conversion, top products) without needing a real data source.

Components

Component	Tool	Role
Event generator	Python (src/generator)	Produces synthetic e-commerce events (page views, add-to-cart, purchases) as JSON lines -- stands in for a real event stream (Kafka/Segment/etc.)
Landing zone	Local filesystem (data/raw)	Immutable raw event storage, one file per batch
Ingestion	Python (src/ingestion)	Reads raw events, validates/normalizes schema, loads into the warehouse (data/warehouse, DuckDB)
Warehouse	DuckDB	Lightweight embedded warehouse -- no server to run, plays well with dbt, good enough for this scale
Transformation	dbt (dbt/)	Staging models (clean/typed events) -> mart models (daily revenue, funnel conversion, top products)
Orchestration	Airflow (dags/)	DAG: generate -> ingest -> dbt run -> dbt test, scheduled daily
Consumption	Streamlit (dashboard/)	Reads mart tables, renders revenue/funnel/product charts

Data flow



src/pipeline.py holds all four steps as functions; the Airflow DAG and the CLI are both thin callers of it.

Event schema (raw)

```
{
  "event_id": "uuid",
  "event_type": "page_view | add_to_cart | purchase",
  "user_id": "string",
  "session_id": "uuid, shared by all events in one browsing session",
  "product_id": "string",
  "product_category": "string",
  "price": "float, present on add_to_cart/purchase",
  "quantity": "int, present on add_to_cart/purchase",
  "timestamp": "ISO-8601 UTC"
}
```

Events are generated in sessions (1-4 events, same user, clustered timestamps) rather than as independent draws -- session_id is what lets funnel_conversion group page_view -> add_to_cart -> purchase per visit instead of per random event. Session starts are spread over a --days window (default 14) so the date-grained marts have more than one date to group by.

Day-by-day plan

Day	Planned	What actually happened
1	Scaffolding, architecture doc, event generator, raw ingestion into DuckDB, tests	as planned
2	Ingestion hardening (schema validation, batching), more realistic event distributions	as planned, plus session grouping
3	dbt project setup + staging models	staging and all three marts -- the staging layer left budget to pull day 4 forward
4	dbt marts + data quality tests	marts re-grained to daily, pipeline module, Airflow DAG (day 5), dashboard (day 6), run auditing
5	Airflow DAG wiring the full pipeline end to end, scheduled	done day 4
6	Streamlit dashboard on top of the marts	done day 4
7	Polish, README pass, docs/retailpulse-report.pdf, tag week01-complete	lessons learned written up below

Days 5 and 6 landed early, so days 5-6 were spent on depth rather than breadth: a dim_products dimension (day 5), backfill/replay from the landing zone (day 5), and the PDF report toolchain plus freshness/volume tests on the pipeline (day 6). Incremental dbt models were considered and rejected -- see the decisions log.

Decisions log

- DuckDB over Postgres: no server process to manage for a solo daily-build project, still real SQL + dbt-compatible, upgrade path to Postgres/Snowflake later is just a dbt profile change.

- Synthetic data over a public dataset: full control over volume/schema/day-to-day evolution, matches the "day 2 hardens ingestion" plan without waiting on external data quirks.
- Airflow over a lighter scheduler (e.g. Prefect): still the most commonly required orchestrator in job postings; worth the extra setup weight for portfolio value.
- In-repo dbt/profiles.yml over ~/.dbt/profiles.yml: keeps the project runnable with a plain git clone + dbt run --profiles-dir ., no machine-specific setup step to document.
- A plain singular test (dbt/tests/assert_*.sql) over adding dbt_utils for the price/quantity range check: one query does the whole job; not worth a package dependency for a single check.
- Marts at daily grain, not all-time (day 4): funnel_conversion and top_products both started as single-row/all-time tables, which can only answer one question and hide every trend they exist to show. At (date, ...) grain any window rolls up downstream from the same table. The cost is that dbt's built-in unique test can't express a composite key, so the grain gets a singular test instead -- still cheaper than a dbt_utils dependency.
- A session is attributed to the date of its first event: a visit crossing midnight is counted once, on the day it began. Splitting it across both dates would double-count the session and make the funnel's denominators disagree with the raw session count.
- The product catalog is seeded independently of event generation (catalog.CATALOG_SEED): it's a reference dimension, not per-batch random data. Seeding it with the run seed meant every batch invented its own product_id -> category mapping, and the date-grained top_products split one product into duplicate rows. Caught by assert_top_products_grain_is_unique on the first real multi-batch run -- the test earned its keep immediately.
- Pipeline logic in src/pipeline.py, not in the DAG: Airflow has no native Windows support and this project is developed on Windows, so a DAG holding the logic would be untestable and unrunnable here. The DAG is wiring; the steps are plain functions with their own tests, and python -m src.pipeline runs the same path without an orchestrator.
- Run auditing in pipeline_runs rather than in ingest(): ingest() loads a file; "which batch ran, when, and how much it rejected" is an orchestration-level fact. Without the table, a batch that quietly rejected half its rows vanished with the run's stdout.
- batch_id in pipeline_runs is not unique: an idempotent retry is supposed to show up as a second row with duplicates == events_read. Making it unique would hide exactly the thing the table is for.
- Dashboard split into app.py (layout) and queries.py (reads): Streamlit rendering is awkward to test, SQL isn't. The logic that can actually be wrong -- window arithmetic, rate derivation, roll-ups -- gets real tests against a built scratch warehouse; app.py gets a render smoke test through Streamlit's own AppTest.
- Dashboard windows anchor on the newest date in the data, not on today: generated batches routinely end a day or two short, and a today-anchored window renders an empty chart.
- profiles.yml reads RETAILPULSE_WAREHOUSE with the old path as default: lets tests and throwaway backfills build models into a scratch database without editing the profile or duplicating it.
- The product dimension is a dbt seed exported from the Python catalog, not a model derived from events (day 5): a select distinct product_id, product_category from stg_events would be a dimension that agrees with the events by construction -- it can't catch anything. Exporting build_catalog() to dbt/seeds/products.csv gives the warehouse an independent reference, so relationships on stg_events.product_id fails when an event names a product the catalog has never heard of. Hand-maintaining the CSV was the other option and was rejected: two copies of the same reference data drift silently.
- dbt seed is its own pipeline step and its own DAG task, between ingest and dbt run: models and tests ref the seed, so it has to be loaded first, and folding it into _dbt_run would hide a step that can fail on its own.
- top_products.product_category is joined from the dimension, not carried by the event (day 5): an event is a claim about what a product is; the catalog is the record. The old assert_product_id_maps_to_one_category test only checked events against each other, so it could never fail in the worst case -- every event for a product agreeing on the same wrong

category. That test was deleted and replaced by `assert_event_category_matches_products`, which compares events to the dimension and is strictly stronger (two categories for one product means at least one disagrees with the catalog). Verified by relabelling all four events for one product: the old check would have passed, the new one fails, and the mart still reports the catalog's category.

- `build_models()` owns the seed-then-run order (day 5): marts ref the products seed, so dbt run on a never-seeded warehouse dies with "Table with name products does not exist". The rule was duplicated across four call sites and was already wrong in one of them -- the dashboard test fixture -- within hours of the seed landing. One function now owns the ordering. The DAG deliberately keeps `dbt_seed` and `dbt_run` as separate tasks anyway, so a seed failure is visible as itself in the Airflow UI rather than folded into a model build.

- Replay lives in `src/pipeline.py`, not in a new module (day 5): a backfill is "for each landing file, oldest first, call the ingest step that already exists". Ingestion has been idempotent since day 2, so replay needed no new loading logic at all -- only iteration order and totals. --replay on the existing CLI rebuilds a deleted warehouse from raw data, which is what makes the append-only landing zone worth having; without it, "immutable landing zone" was a claim the project never cashed in.

- Replay's audit rows are dated when the replay ran, not when the batch originally landed: `pipeline_runs` is a record of ingests, and a replay genuinely is a new ingest. Backdating it would make the audit table lie about what happened. The original `ingested_at` is lost with the warehouse -- accepted, since the landing zone (not the audit table) is the source of truth being replayed.

- Marts stay full-refresh; incremental models were considered and rejected (day 5): the whole warehouse is a few thousand rows in DuckDB and rebuilds in under a second. Incremental would buy nothing measurable and cost real complexity -- a `unique_key`, a lookback window for late-arriving events, and a --full-refresh path that has to be remembered whenever a model changes. Incremental models are also a common source of silently stale marts. Revisit if a full rebuild ever becomes slow enough to notice.

- `tests/test_catalog.py` asserts the committed CSV byte-matches a fresh export: the seed is generated but checked in, so the realistic failure is someone editing it by hand or changing the catalog without re-exporting. Catching that in pytest beats discovering it as a relationships failure on production-shaped data.

- `docs/generate_report.py` uses `fpdf2`, not a headless-browser/Pandoc toolchain (day 6): the day-7 deliverable is one document from one Markdown source, so a minimal pure-Python renderer beats a new system dependency. Two `fpdf2` pitfalls worth remembering: `multi_cell`'s default `new_x` leaves the cursor at the end of the rendered text rather than the left margin, so back-to-back calls starve each other of width unless `new_x=XPos.LMARGIN` is passed explicitly; and the core Helvetica/Courier fonts only support latin-1, not cp1252, so the doc's em-dashes and arrows need transliterating before render rather than assuming any 8-bit codepage will do.

- Volume and freshness are two separate singular tests, not one (day 6): a run that reads a batch and legitimately loads nothing (an empty source) looks identical to a stalled pipeline (no new batches at all) if you only check `pipeline_runs`, and a warehouse can be voluminous yet stale if ingestion stops but nothing deletes old rows. `assert_latest_run_loaded_something` catches the first by checking the latest run's counts; `assert_stg_events_not_stale` catches the second by checking the newest `event_at` against a `freshness_window_days` var (default 14, matching the generator's own --days window) rather than the run log. Verified independently: the volume test via an injected zero-everything row, the freshness test via --vars `freshness_window_days:0`.

- The freshness guardrail immediately caught a real problem (day 6): the local warehouse hadn't been touched since day 5 -- about six weeks of real elapsed time -- so `assert_stg_events_not_stale` failed against it on the very first run, correctly. The fix was to actually run the pipeline again, not to loosen the threshold; a test that can be satisfied by widening its own window until it passes isn't testing anything.

- No DAG or `pipeline.py` change needed for either new guardrail: both are dbt data tests, and `run_dbt()` already raises on any non-zero dbt test exit code, which already fails `run_pipeline()`/the DAG's `dbt_test` task. Wiring a new failure mode

into an already-general failure path would have been work with no behavior change.

- The generic tests were left in shorthand form; the deprecation warning noted on day 6 does not reproduce (day 7): day 6 flagged a `MissingArgumentsPropertyInGenericTestDeprecation` seen during a dbt test run and left rewriting the generic tests under an `arguments: key as optional` day-7 work. On the pinned dbt-core 1.12.0, `dbt parse --show-all-deprecations` and `dbt parse --warn-error` are both clean, and no deprecation appears in a full dbt test either. Rewriting 30-odd test declarations to silence a warning this version doesn't emit would be churn with no verifiable effect, so `_seeds.yml/_staging.yml/_marts.yml` keep the shorthand. Worth revisiting on a dbt upgrade, where the warning would become real and reproducible first.

- The report renderer draws Markdown tables with fpdf2's own table API, not hand-measured columns (day 7): the first real render of this document put the Components and day-by-day tables into the PDF as raw pipe text, wrapping mid-row, `|---|---` separator rows included. fpdf2 already measures text and splits the available width, so the fix was a branch that collects consecutive `|` lines, drops the separator row, and hands the rest to `pdf.table()`. The fence check has to come first: the data-flow diagram above is drawn with pipes inside a code block, and several of its lines begin with one, so a table branch evaluated earlier would parse the diagram into a mangled table. That ordering has its own test.

- The stack appendix is rendered from `requirements.txt`, not restated in the prose (day 7): the day-7 report should state what the project runs on, and a reader of the PDF has no `requirements.txt` to check a hand-written version table against. Generating the appendix from the file means the two cannot disagree -- the same argument that made the product dimension an export rather than a hand-maintained CSV.

- The renderer no longer strips `_` as emphasis (day 7): it had since day 6, and reading the first real report end to end showed what that costs a document written almost entirely in `snake_case` -- `session_id` printed as "sessionid", `CATALOG_SEED` as "CATALOGSEED", `funnel_conversion` as "funnelconversion", `dbt_utils` as "dbtutils", every occurrence, for six pages. This document uses no `_emphasis_` anywhere (52 asterisk emphases, zero underscore ones), so not stripping underscores trades away nothing. Asterisks are still stripped, but no longer inside a code span, because `assert_*.sql` is a glob and was being printed as `assert.sql`. The lesson generalises past this script: an inline-Markdown stripper written for prose quietly corrupts identifiers, and the corruption is invisible unless something reads the rendered output back.

- The report PDF is committed and its creation date is pinned (day 7): the day-7 deliverable is checked in, so it has to behave like a file under version control. fpdf2 stamps the current time into every PDF, which made the committed binary byte-different on every regeneration -- a diff that says nothing about whether the report changed. `set_creation_date` with a constant makes the output reproducible, so a diff on `docs/retailpulse-report.pdf` means the content moved. Same reasoning as pinning the exported seed to LF in `.gitattributes`: a generated file that is also committed needs to be deterministic or it is permanently dirty.

- pypdf is pinned as a test dependency so the report can be asserted on, not just weighed (day 7): the existing smoke test checked that a %PDF file over 1 KB was written, which passes just as happily on a report full of mangled tables or ? characters -- it did, for one commit. Extracting the text and asserting on it is the only way to catch a rendering regression, and the dependency is test-only. pytest is already pinned in `requirements.txt`, so there is no dev/prod split to respect.

Known limitations

Stated plainly, because the components table above lists tools the project does not exercise to the same depth.

- The Airflow DAG has never been executed. Airflow has no native Windows support and this project is developed on Windows, so `dags/retailpulse_pipeline.py` is structurally validated only: `tests/test_dag.py` checks task ids, the `>>` chain order, and that every `pipeline.*` function the DAG calls still exists, all at AST level so they run anywhere. The one test

that would import it through Airflow (DagBag) is skipped here and runs only where Airflow is installed. Everything the DAG orchestrates does run, via `python -m src.pipeline`, which is the same code path.

- The data is synthetic. `src/generator` stands in for a real event stream; there is no Kafka, no CDC, no API ingestion, and no late-arriving or malformed traffic beyond what the generator is told to produce. Ingestion's validation and quarantine path is exercised by deliberately invalid fixtures rather than by real-world mess.
- Scale is deliberately small. A few thousand rows in a single-file DuckDB warehouse that rebuilds in under a second. That is what makes full-refresh marts the right call here (see the decisions log) and it is also why none of the usual large-data concerns -- partitioning, incremental loads, warehouse cost -- were engineered for.
- DuckDB allows one writer. Hence `max_active_runs=1` on the DAG. Concurrent ingests are a design constraint of this warehouse choice, not something the pipeline handles.
- No CI. The pytest suite and the dbt data tests are run locally; nothing enforces them on push. A GitHub Actions workflow running pytest plus a dbt build against a scratch warehouse is the obvious next step and would have caught the `core.autocrlf` seed-drift bug (day 5) on the first clean checkout.
- The dashboard is local-only. `streamlit run dashboard/app.py` against the local warehouse file; there is no deployment, no auth, and no shared/remote warehouse behind it.

Lessons learned

Each of these cost real debugging time during the week. They're grouped by what they generalise to rather than by date, with the day that produced each one noted so it can be traced back to the commits.

- A test that compares the data to itself cannot fail in the worst case (day 5). `assert_product_id_maps_to_one_category` checked events against other events, so a product whose every event agreed on the same wrong category passed cleanly. Replacing it with a check against the independently-seeded dimension made it strictly stronger. When writing a data test, ask what independent source it is checking against -- if the answer is "the same table", it is checking consistency, not correctness.
- Reference data has to be seeded independently of the random data that references it (day 4). `build_catalog()` took the run seed, so every batch invented its own `product_id` -> category mapping and one product became duplicate rows in a date-grained mart. A dimension is not per-batch random data, and giving it its own fixed seed is the whole fix.
- An append-only landing zone is a claim until something reads it back (day 5). `data/raw/` had been immutable since day 4, but a deleted warehouse still meant regenerating different synthetic data, so the design bought nothing. Writing --replay needed no new loading logic -- ingestion had been idempotent since day 2 -- which is the tell that the capability was already paid for and simply never wired up.
- An ordering rule duplicated across call sites is already wrong in one of them (day 5). "Seed before run" lived in four places and the dashboard test fixture had it wrong within hours of the seed landing. Moving it into `build_models()` fixed the bug and the class of bug together.
- A freshness failure is usually telling the truth (day 6, again on day 7). `assert_stg_events_not_stale` failed on a warehouse untouched for six weeks, and failed again on day 7 after another two. Both times the fix was running the pipeline, not widening the window. A test that can be satisfied by relaxing its own threshold has stopped being a test.
- Platform assumptions leak into tests, not just code (day 5). The byte-comparison guard on the exported seed passed locally and would have failed on every fresh clone, because `core.autocrlf` rewrites line endings on checkout while the exporter writes LF. Cloning the repo was the only thing that surfaced it -- the guard was green in the working copy the whole time.
- Derived rates must be recomputed from totals, never averaged across groups (day 4). Averaging per-day conversion rates weights a quiet day the same as a busy one, and the result visibly contradicted the totals printed beside it on the

dashboard.

- Pick the dependency the deliverable actually needs (day 6). The day-7 report is one document from one Markdown source, so fpdf2 beat a headless browser or Pandoc. The cost was two library-specific pitfalls -- multi_cell leaving the cursor at the end of the text rather than the left margin, and the core fonts being latin-1-only -- both cheap next to a system dependency.

Appendix: pinned stack

Verbatim from requirements.txt, so this page cannot drift from what the project actually installs.

Package	Version
duckdb	1.0.0
dbt-duckdb	1.8.1
dbt-core	1.12.0
apache-airflow	2.9.3
streamlit	1.60.0
pandas	2.2.2
pyarrow	17.0.0
faker	26.0.0
pytest	8.3.2
fpdf2	2.8.8
pypdf	6.14.2